The Largest Binary Four-Deletion-Correcting Code of Length Twelve

Anonymous

September 28, 2026

Abstract

A binary code of length n corrects t deletions when no two codewords share a subsequence of length $n - t$. Let $L_2(n, t)$ be the largest size of such a code. For length twelve and four deletions, Kolev's 2016 table records $6 \le L_2(12, 4) \le 10$, and a 2008 table of variable-length insertion/deletion codes by Baker, Flack, and Houghten implies $L_2(12, 4) \le 7$. We prove

$$L_2(12, 4) = 6.$$

A six-word code gives the lower bound. For the upper bound we construct an exact weighted-packing certificate: 117 nonzero weights on length-eight descendant rows and 20 weighted cliques of the conflict graph. Each of the 4,096 binary words of length twelve receives weight at least one, and the weights sum to $6.999616 < 7$, so no code has seven words. A standard-library Python program checks the certificate in integer arithmetic. A Lean 4 development proves the weighted-sum argument and checks the certificate data.

1 Introduction

A deletion channel drops symbols and does not tell the receiver where. Levenshtein founded the theory of codes that survive such errors [Lev66]. A deletion shifts every later symbol, so the Hamming-distance tools of classical coding theory do not apply, and the exact extremal numbers remain unknown even for short binary words.

The finite values of $L_2(n, t)$ have a short history. Landjev and Haralambiev tabulated lower and upper bounds for $t = 2, 3, 4, 5$ in 2007 [LH07]; at $(n, t) = (12, 4)$ their tables give $5 \le L_2(12, 4) \le 10$. Kolev's 2016 table raises the lower end to six and credits that value to a 2015 Sofia University master's thesis [Kol16, Table 1], [Lan15]. A sharper upper bound follows from a neighbouring problem. Baker, Flack, and Houghten computed optimal variable-length insertion/deletion codes, in which n is the length of the longest codeword, and their table lists $D_2(12, 10) = 7$ [BFH08, Hou08]. Two words of length twelve are at insertion/deletion distance $24 - 2\,\mathrm{LCS}(x, y)$, so a four-deletion-correcting code of length twelve has minimum distance at least ten and counts as a variable-length code. Hence $L_2(12, 4) \le 7$, although no fixed-length table records this. Recent computational work concentrates on fewer deletions: Nakasho, Hagiwara, Anderson, and Nation proved $L_2(11, 1) = 172$ with integer programming [NHAN23], and Weindel and Heckel tabulate bounds for two and three deletions up to length eighteen [WH26]. Neither work treats four deletions. Table 1 summarizes the record at $(12, 4)$.

Theorem 1.1 (Main theorem)**.** *The largest binary code of length twelve that corrects four deletions has six codewords. Equivalently, among any seven binary words of length twelve, some two share a common subsequence of length eight.*

Source	Lower bound	Upper bound
Landjev and Haralambiev (2007) [LH07]	5	10
Baker, Flack, and Houghten (2008), via $D_2(12, 10)$ [BFH08]		7
Kolev (2016) [Kol16]	6	10
This paper	6	**6**

Table 1: Published bounds on $L_2(12, 4)$.

The six-word code in section 3 reproduces the known lower bound. The new content is the upper bound: seven codewords cannot coexist, which improves the implied bound of seven by one. The argument has three parts.

1. A weighted-packing lemma (lemma 4.1) combines the standard descendant-row constraints with inequalities from cliques of the conflict graph.

2. An explicit certificate (section 5) supplies nonnegative integer weights whose normalized total is 6.999616.

3. Two independent checkers, a standard-library Python program and a Lean 4 development, verify every coefficient, coverage inequality, and clique pair.

We found the certificate with floating-point linear programming, but the theorem depends only on the exported integers and one weighted sum. The proof inherits nothing from the solver.

2 Definitions and the conflict hypergraph

Let $[q] = \{0, 1, \ldots, q-1\}$. For $x = x_1 x_2 \cdots x_n \in [q]^n$ and $0 \le t < n$, define

$$\mathcal{D}_t(x) = \{x_{i_1} x_{i_2} \cdots x_{i_{n-t}} : 1 \le i_1 < i_2 < \cdots < i_{n-t} \le n\}.$$

Different choices of deleted positions can produce the same word; that word counts once in $\mathcal{D}_t(x)$.

Definition 2.1. A code $\mathcal{C} \subseteq [q]^n$ corrects t deletions if

$$\mathcal{D}_t(x) \cap \mathcal{D}_t(y) = \varnothing \qquad \text{for all distinct } x, y \in \mathcal{C}.$$

Let $L_q(n, t)$ denote the maximum cardinality of such a code.

Some authors write $A_q(n, t)$ for the same quantity. We follow the notation of the finite tables in [LH07, Kol16].

Fix (q, n, t) and form a hypergraph on the vertex set $[q]^n$. Each $d \in [q]^{n-t}$ indexes a row

$$R_d = \{x \in [q]^n : d \in \mathcal{D}_t(x)\}.$$

A deletion-correcting code meets each R_d in at most one vertex. This set-packing formulation underlies the non-asymptotic upper bounds of Kulkarni and Kiyavash [KK13].

The *conflict graph* G has vertex set $[q]^n$, with two distinct words adjacent when they share a deletion descendant. Codes correcting t deletions are the independent sets of G, so a code meets any clique $K \subseteq V(G)$ in at most one word.

For equal-length words x, y, let $\mathrm{LCS}(x, y)$ be the length of a longest common subsequence.

Lemma 2.2. *For $x, y \in [q]^n$, the sets $\mathcal{D}_t(x)$ and $\mathcal{D}_t(y)$ intersect if and only if* $\mathrm{LCS}(x, y) \geq n - t$.

Proof. A common descendant is a common subsequence of length $n - t$. Conversely, a common subsequence of length at least $n - t$ contains one of length exactly $n - t$, and that word lies in both descendant sets. □

3 A code of size six

Consider the six binary words

$$\begin{aligned} \mathcal{C}_0 = \{&000000000000, \quad 000011111110, \quad 010101010101, \\ &101111100000, \quad 110000000111, \quad 111111111111\}. \end{aligned} \tag{1}$$

Their pairwise longest-common-subsequence lengths, in the order listed, are:

	1	2	3	4	5	6
1	12	5	6	6	7	0
2	5	12	7	7	7	7
3	6	7	12	7	7	6
4	6	7	7	12	7	6
5	7	7	7	7	12	5
6	0	7	6	6	5	12

Each off-diagonal entry is at most $7 < 12 - 4$, so by lemma 2.2 the four-deletion descendant sets are pairwise disjoint.

Proposition 3.1. $L_2(12, 4) \geq 6$.

Proof. The six distinct words in (1) form a four-deletion-correcting code by the table above. □

The released checker confirms this claim two ways. It enumerates the four-deletion descendants of each word by keep/delete recursion, and it computes all 15 LCS values by dynamic programming. The six descendant sets contain 198 distinct words in total, with no collision between codewords.

4 A weighted certificate lemma

The upper bound puts nonnegative weights on two families of packing inequalities. We state the argument in the form the certificate uses.

Lemma 4.1 (Weighted descendant-and-clique bound)**.** *Fix q, n, t. Let $a_d \geq 0$ for $d \in [q]^{n-t}$. Let $K_1, \ldots, K_m$ be cliques in the conflict graph, with weights $b_i \geq 0$. Suppose each $x \in [q]^n$ satisfies*

$$\sum_{d \in \mathcal{D}_t(x)} a_d + \sum_{i : x \in K_i} b_i \geq 1. \tag{2}$$

Then each t-deletion-correcting code $\mathcal{C} \subseteq [q]^n$ satisfies

$$|\mathcal{C}| \leq \sum_{d \in [q]^{n-t}} a_d + \sum_{i=1}^{m} b_i. \tag{3}$$

Proof. Sum (2) over $x \in \mathcal{C}$ and exchange the order of summation:

$$\begin{aligned} |\mathcal{C}| &\le \sum_{x \in \mathcal{C}} \left(\sum_{d \in \mathcal{D}_t(x)} a_d + \sum_{i : x \in K_i} b_i \right) \\ &= \sum_d a_d |\mathcal{C} \cap R_d| + \sum_i b_i |\mathcal{C} \cap K_i|. \end{aligned}$$

The code meets each row R_d and each clique K_i in at most one word. Nonnegativity of the weights gives (3). □

The clique terms carry the proof. With weights on the 256 descendant rows alone, the best bound this lemma can give is the optimum of the fractional packing LP, which HiGHS reports as about 7.054. That yields only $L_2(12, 4) \le 7$, the same bound the variable-length table implies. Adding 20 cliques pushes the bound below seven, and each clique remains easy to check: every pair of its words must share a length-eight subsequence.

5 The exact certificate

Set $M = 1{,}000{,}000$. The certificate stores integers $A_d \ge 0$ for $d \in \{0,1\}^8$ and $B_i \ge 0$ for 20 conflict cliques K_i, and sets

$$a_d = A_d / M, \qquad b_i = B_i / M.$$

The JSON file omits zero descendant weights. The checker verifies the following facts in exact arithmetic.

Quantity	Verified value
Length-12 words checked	4,096
Possible length-8 descendants	256
Nonzero descendant weights	117
Conflict cliques	20
Clique sizes	62 to 94
Clique pairs checked	61,263
Minimum integer word coverage	1,000,000
Sum of all integer weights	6,999,616

So for each $x \in \{0,1\}^{12}$,

$$\sum_{d \in \mathcal{D}_4(x)} A_d + \sum_{i : x \in K_i} B_i \ge M, \tag{4}$$

while

$$\sum_d A_d + \sum_i B_i = 6{,}999{,}616 < 7M. \tag{5}$$

Proposition 5.1. $L_2(12, 4) \le 6$.

Proof. Divide (4) and (5) by M and apply lemma 4.1. Each valid code $\mathcal{C}$ satisfies

$$|\mathcal{C}| \le \frac{6{,}999{,}616}{1{,}000{,}000} = 6.999616 < 7.$$

Since $|\mathcal{C}|$ is an integer, $|\mathcal{C}| \le 6$. □

Propositions 3.1 and 5.1 together prove the main result.

Theorem 5.2. *In the ordinary ordered binary deletion model,*

$$\boxed{L_2(12, 4) = 6}.$$

6 Finding and checking the certificate

We separate the search, which no one needs to trust, from the small computation that the proof relies on.

6.1 Search

We treated the 4,096 words of length twelve as vertices of the conflict graph and solved the linear-programming relaxation of the 256 descendant-row packing constraints with HiGHS. While the LP optimum stayed at seven or above, we solved an integer maximum-weight clique problem on the support of the current LP solution to find a violated clique inequality, then added it as a cut. After 20 cuts the optimum fell below seven. We multiplied the nonnegative dual multipliers by M, rounded each one up, and kept the rounded vector only after the exact coverage and total checks passed. Theorem 5.2 uses none of the floating-point output.

6.2 Standalone checker

The verifier is a Python program that imports only the standard library. Given the JSON certificate, it checks that:

1. the declared model and parameters equal $(q, n, t) = (2, 12, 4)$;
2. each word and descendant is a fixed-width binary string;
3. each clique lists distinct words, and all 61,263 within-clique pairs share a four-deletion descendant, reconstructed from scratch;
4. each of the 4,096 words satisfies (4);
5. the stored total equals 6,999,616, below $7M$; and
6. the certificate bytes match the published SHA-256 digest.

A separate program, independent of the search code, checks the six-word code through both descendant-set disjointness and the LCS criterion. Mutation tests lower a necessary weight and corrupt a clique, and the verifier rejects both.

Artifact	SHA-256
Codebook	`5dee9d5e15202ebe6ecff0fe0f35e704` `e3b8d957967a9116becdeaa6cc1c023f`
Upper certificate	`d9c8b720d112957250c3c06ec786634a` `573d6e1f78a449f2b2da5a829cf7fc82`

7 Lean 4 formalization

The Lean development defines the ordered deletion model, imports the certificate as transparent data, checks its finite facts, and proves the weighted-sum argument. Words of length twelve are elements of `Fin 4096` and descendants are elements of `Fin 256`. The predicate `hasDescendant` compares fixed-width bit lists with `List.isSublist`, which tests for a subsequence. A family `code : Fin k -> Word` is `ValidFamily` when it is injective and no two of its words share a length-eight descendant.

The main theorem reads:

```
theorem binary_length12_four_deletion_exact :
    (lowerCode.Nodup /\ lowerCode.length = 6 /\
      explicitCodeValid lowerCode = true) /\
    Not (Exists fun code : Fin 7 -> Word => ValidFamily code)
```

The second conjunct excludes every injective labelled family of seven compatible words. A code with seven or more words contains such a family, so the two conjuncts together state theorem 5.2. The proof isolates four finite checks from the structural argument:

- validity and size of the six-word code;
- validity of the 20 conflict cliques;
- coverage of all 4,096 words; and
- the exact total-weight inequality.

The development contains no `sorry`, no `admit`, and no project-local axiom. Its four finite checks use `native_decide`, so the Lean proof trusts Lean's compiler and runtime in addition to its kernel; `#print axioms` lists exactly these four leaves beyond the standard axioms. Replacing them with kernel-checked proof terms would remove that dependence. The Python checker does not share it.

8 Reproducibility

From the `formalization` directory of the accompanying repository, run:

```
python3 research/deletion_code_attempt/verify_code.py
python3 research/deletion_code_attempt/proof/verify_upper_certificate.py
python3 research/deletion_code_attempt/proof/test_upper_certificate_mutations.py
lake build DeletionCodes.Conjectures.DeletionCode124
lake build
```

The Python programs end with these markers:

- `VERIFICATION=PASS` for the six-word code;
- `UPPER_CERTIFICATE=PASS` for the upper certificate; and
- `MUTATION_AUDIT=PASS` for the mutation tests.

The Lean build prints the axioms of the main theorem, so a reader can inspect its trust boundary.

9 Concluding remarks

This paper settles one finite entry, not an asymptotic question. A reader can check the whole proof: six lines give the construction, one weighted sum gives the upper bound, and a versioned certificate holds every finite fact, checked in both Python and Lean. The same combination of descendant rows and clique cuts applies without change to other open entries. The nearest are $6 \le L_2(13, 4) \le 14$ and $7 \le L_2(14, 4) \le 22$ in Kolev's table, which lie beyond the range of the variable-length tables.

Literature search. Searches through September 28, 2026 found the bounds in table 1 and no source that states the exact value or an upper bound below seven. Recent finite-length studies stop at three deletions, and arXiv listings from July through September 2026 contain no competing result. We could not obtain the 2015 thesis that Kolev cites, and an unpublished result could exist. The theorem and its certificate stand independent of any priority question.

Use of AI tools. The author used AI coding and reasoning assistants to write search code, the certificate checkers, the Lean formalization, and drafts of this manuscript. The author takes responsibility for the statements, the literature claims, the released artifacts, and the correctness of the paper.

References

[BFH08] S. Baker, R. Flack, and S. Houghten. Optimal variable-length insertion-deletion correcting codes and edit metric codes. *Congressus Numerantium*, 186:65–80, 2008.

[Hou08] Sheridan Houghten. Optimal insertion-deletion correcting codes. `https://www.cosc.brocku.ca/~houghten/insdel.html`, 2008. Table of bounds, last modified August 5, 2008.

[KK13] Ankur A. Kulkarni and Negar Kiyavash. Nonasymptotic upper bounds for deletion correcting codes. *IEEE Transactions on Information Theory*, 59(8):5115–5130, 2013.

[Kol16] Emil Kolev. Optimal binary t-deletion-correcting codes. In *Proceedings of the Forty-Fifth Spring Conference of the Union of Bulgarian Mathematicians*, pages 135–139, Pleven, Bulgaria, 2016.

[Lan15] K. Landgev. Deletion-correcting codes. Master's thesis, Sofia University, 2015. Cited by Kolev (2016) as K. Landgev, *Deletion-Correcting Codes*, and in arXiv:1607.01233 as K. Landjev, *Codes Correcting Deletions*; not available online.

[Lev66] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10:707–710, 1966. English translation of Doklady Akademii Nauk SSSR 163 (1965), 845–848.

[LH07] Ivan Landjev and Kristiyan Haralambiev. On multiple deletion codes. *Serdica Journal of Computing*, 1(1):13–26, 2007.

[NHAN23] Kazuhisa Nakasho, Manabu Hagiwara, Austin Anderson, and J. B. Nation. The tight upper bound for the size of single deletion error correcting codes in dimension 11, 2023.

[WH26] Franziska Weindel and Reinhard Heckel. LLM-guided search for deletion-correcting codes. *Transactions on Machine Learning Research*, 2026.